

MUSEO DE BALER · BALER, AURORA

System User Manual

Visitor and Operations System

A complete guide to operating and evaluating the Museo de Baler system: the staff panel that runs the museum day to day, the progressive web app visitors open on their own phones, and the JSON API that joins the two.

Part A is written for the people who use the system.

Part B is written for technical evaluators.

Prepared for the technical panel and IT evaluators

Document version 1.0 · 29 September 2026

Software release: Laravel 12 on PHP 8.3, MySQL 8

Deployed at <https://museodebaler.com>

About this manual

This manual documents one system with three faces. Read the part that matches what you are doing; the two parts are written to be read independently.

Who each part is for

Part	Written for	What it answers
Part A Operating the system	Museum staff, the Municipal Tourism Office, and anyone following a live demonstration	How a visitor is admitted and guided, how the collection is maintained, how reports are produced, and what to do when a screen shows an error. No technical background assumed.
Part B Technical reference	IT evaluators, system administrators, and any developer who inherits the code	The stack, architecture, request pipeline, data model, API surface, security controls, installation, deployment, backup and restore, verification, and the boundaries drawn on purpose.
Appendices	Both	Command reference, directory map, pre-production checklist, glossary.

Conventions

- **“The panel”** means the website staff sign in to on a computer or tablet. **“The app”** means what a visitor opens on their own phone. They are the same software and the same database, reached by two different doors.
- `Monospaced text` is something typed exactly as shown: a command, a file path, a configuration key, or a URL.
- **Bold** inside an instruction is a button or a menu item on the screen.
- Figures are screenshots of the running system, not mock-ups.

Figures in this manual

Every screenshot in Part A was taken from the working system. Where the manual and the screen disagree, the screen is correct and this document is out of date — report it to whoever maintains the system so the next revision is right.

Document control

Item	Value
Document	Museo de Baler — System User Manual
Version	1.0
Date	29 September 2026
Subject system	Museo de Baler visitor and operations system
Software stack	Laravel 12, PHP 8.3 (8.2 minimum), MySQL 8, vanilla-JavaScript PWA
Production address	https://museodebaler.com
Verification at time of writing	406 PHPUnit tests (4,902 assertions) and 15 service-worker tests, all passing

Item	Value
Companion documents in the repository	docs/ARCHITECTURE.md, docs/ERD.md, docs/SECURITY.md, docs/DEPLOYMENT.md, docs/HOSTING.md, docs/RESTORE.md, docs/OWNERSHIP.md, docs/LIMITATIONS.md, docs/STAFF_MANUAL.md

One idea to understand before anything else. A visitor's phone receives an access token when they register, but exhibit content is released only after the front desk has collected the admission fee or sighted a Baler resident's identification. That decision is made on the server, on every single request. The app's “waiting for the desk” screen is a rendering of the server's answer, never the check itself. Almost everything else in this manual follows from that sentence.

About this manual	2
<i>Who each part is for</i>	2
<i>Conventions</i>	2
<i>Figures in this manual</i>	2
<i>Document control</i>	2
A1 · What the system is	8
<i>A1.1 The addresses</i>	8
<i>A1.2 What replaces what</i>	8
A2 · Signing in, and who can do what	9
<i>A2.1 The two roles</i>	9
A3 · The visitor's journey on a phone	11
<i>A3.1 The sequence</i>	11
<i>A3.2 Settings the visitor controls</i>	11
<i>A3.3 What the visitor keeps when the signal drops</i>	12
A4 · The front desk	13
<i>A4.1 Three ways a visitor gets in</i>	13
<i>A4.2 Clearing a visitor</i>	14
<i>A4.3 Groups: schools, families, tour agencies</i>	14
<i>A4.4 Refunds and corrections</i>	14
<i>A4.5 End of day</i>	14
A5 · Exhibits: add, edit, archive	15
<i>A5.1 Adding an exhibit</i>	15
<i>A5.2 Editing an exhibit</i>	16
<i>A5.3 Archive and restore</i>	16
A6 · Translations and the audio guide	17
<i>A6.1 Method A: let the AI draft them</i>	17
<i>A6.2 Method B: type a translation and upload a recording</i>	18
A7 · QR codes and the entrance poster	19
A8 · The museum map and the storyline	20
A9 · Museum info: hours, fee, halls, the geofence	21
<i>A9.1 The fields</i>	21
<i>A9.2 The geofence</i>	21
A10 · Camera recognition	23
A11 · Guided tours and visitor feedback	24
<i>A11.1 Tours</i>	24
<i>A11.2 Feedback and the ARTA survey</i>	24
A12 · The Tourism office's screens	25
<i>A12.1 Staff accounts</i>	25
<i>A12.2 Staff attendance</i>	25
<i>A12.3 Attendance corrections</i>	25
<i>A12.4 The visitor survey</i>	25
A13 · Reports and exports	26
<i>A13.1 The six reports and their formats</i>	26
<i>A13.2 The audit trail is Tourism-only</i>	26
A14 · Troubleshooting	27
<i>A14.1 When a visitor asks for help</i>	27
<i>A14.2 When the panel shows an error</i>	28
A15 · Routine maintenance	29
<i>A15.1 Weekly, whoever opens the museum</i>	29
<i>A15.2 Quarterly, the Tourism office with the maintainer</i>	29
B1 · Stack and inventory	31
<i>B1.1 Platform</i>	31
<i>B1.2 Code inventory</i>	31
<i>B1.3 What is authored and what is generated</i>	32
B2 · Architecture	33

B2.1 The three faces	33
B2.2 External dependencies, both optional	33
B3 · The request pipeline	34
B3.1 The panel (session-authenticated)	34
B3.2 The visitor API	34
B4 · Authentication and authorisation	36
B4.1 Two populations, two guards, two tables	36
B4.2 Role enforcement	36
B4.3 The admission gate	37
B5 · The visitor API	38
B5.1 Endpoint reference	38
B5.2 Media is served through short-lived signed URLs	38
B5.3 Design note on API shapes	39
B6 · The data model	40
B6.1 The tables, by the part of the museum they serve	40
B6.2 Conventions	40
B6.3 Three places a foreign key is absent on purpose	40
B7 · Three flows worth tracing	42
B7.1 A visitor checks in at the door	42
B7.2 A visitor identifies an exhibit by camera	42
B7.3 Tourism exports the ARTA report	42
B8 · Offline behaviour	44
B9 · Security controls	45
B9.1 Input and output	45
B9.2 Browser and API hardening	45
B9.3 Logging and alerting	46
B9.4 Data at rest and in transit	46
B9.5 Dependencies and delivery	46
B9.6 Security review record	46
B10 · Installing it locally	48
B11 · Configuration reference	49
B12 · Deployment, rollback and CI	50
B12.1 The layout on the server	50
B12.2 What a deploy does, and where it stops	50
B12.3 Rolling back	50
B12.4 What CI guarantees before any of that	51
B12.5 The scheduler must be woken by the operating system	51
B13 · Backup and restore	52
B13.1 What runs nightly	52
B13.2 The commands	52
B13.3 There is deliberately no db:restore command	52
B13.4 Restore drills	52
B14 · Verification	53
B14.1 What the suites cover	53
B15 · Known limitations and design boundaries	54
B15.1 Deliberate boundaries	54
B15.2 Things a developer will trip over	54
B16 · Data privacy (RA 10173)	55
Appendix A · Command reference	57
A.1 Project commands	57
A.2 Everyday framework commands	57
Appendix B · Directory map	58
Appendix C · Pre-production checklist	59
Appendix D · Glossary	60
Appendix E · Further documentation in the repository	61

P A R T A

Operating the System

What each person does, screen by screen: admitting visitors at the door, maintaining the collection, guiding visitors through the museum on their own phones, producing the reports the LGU files, and recovering when something goes wrong.

A1 · What the system is

Museo de Baler runs as one application with three faces. They share one database, so a fee collected at the desk is visible on the visitor's phone a second later, and a scan made in a hall appears in the evening's report.

Face	Who uses it, and where	What it does
The staff panel	Museum staff and the Municipal Tourism Office, on an office computer or the counter tablet	The front desk, the exhibit collection, translations and narration, QR labels, the museum map, guided tours, staff attendance, visitor feedback, and every report
The visitor app	Visitors, on their own phones	Register, wait for the desk to clear them, then scan or photograph exhibits, read and listen in their chosen language, follow the storyline or roam freely, and leave feedback
The API	Nobody directly — the app talks to it	Serves the app its data as JSON and enforces the admission gate on every request

A1.1 The addresses

Who	Where they go
Visitors	https://museodebaler.com — the bare domain opens the visitor app
Visitors, from a display case	https://museodebaler.com/visitor/?scan=EXH-2026-001 — what a QR label encodes
All staff, both roles	https://museodebaler.com/login

There is no separate Tourism website. Both roles sign in at the same address; what each one sees afterwards is decided by their role, so the head of tourism never sees the exhibit editor and museum staff never see account creation or the audit trail.

A1.2 What replaces what

- The **paper logbook** at the entrance becomes the Front Desk screen, which also prices admission, handles a whole party with one payment, and prints the day for the cash count.
- **Printed exhibit labels in one language** become an exhibit record carrying a title, description, fun facts and spoken narration in English, Filipino and Spanish.
- The **paper daily time record** becomes a clock-in by phone from inside the museum grounds, with a monthly DTR the Tourism office prints.
- The **ARTA Client Satisfaction survey** on paper becomes a sheet in the app, with the CSM figures and the CSV the office files with the LGU produced from the answers.

A2 · Signing in, and who can do what

Open the panel's address in a browser **on a computer or tablet**. The panel is laid out for a screen that size; on a phone it deliberately narrows to clocking in and changing your own password, which are the only two things that need one.

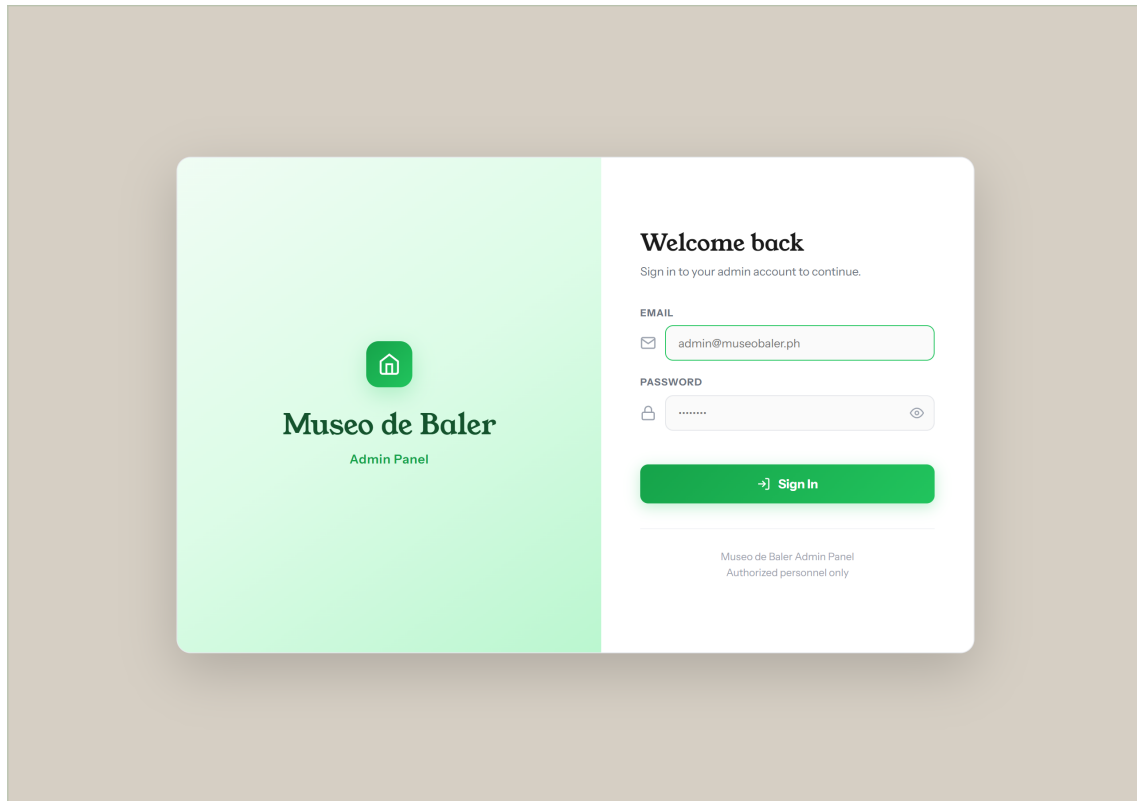


Figure A2.1 — The staff sign-in screen. The same address serves both roles.

1. Enter your email address and the password you were issued.
2. **The first time, you must choose a new password.** It must be at least 12 characters with upper and lower case, a number and a symbol, and may not contain your name or the museum's name.
3. After repeated wrong attempts the system pauses sign-in briefly. If you have forgotten your password, the Tourism office issues a new one (section A12).

A2.1 The two roles

There are exactly two kinds of staff account. The split is deliberate, and it is enforced by the server on every request rather than by hiding menu items.

Can do	Administrator (museum staff)	Tourism Head (Municipal Tourism Office)
Front desk: register visitors and groups, collect fees, verify IDs	Yes	—
Exhibits, translations, narration, QR codes, gallery images	Yes	—
The museum map and the storyline path	Yes	—
Museum info: hours, admission fee, halls, the geofence	Yes	—

Can do	Administrator (museum staff)	Tourism Head (Municipal Tourism Office)
Camera recognition: training photos and training the model	Yes	—
Guided tours	Yes	—
Clock in and out; file an attendance correction	Yes	—
Visitor records, feedback, activity log, staff attendance sheet	Yes	Yes
Reports: logbook, DTR, visitors, exhibits, feedback	Yes	Yes
Create staff accounts and reset their passwords	—	Yes
Staff work schedules	—	Yes
Approve or reject attendance corrections	—	Yes
Edit the visitor survey question bank	—	Yes
Export the audit trail	—	Yes

Why the split is drawn here. The museum is staffed by the LGU tourism office. If museum staff could create accounts, the tier being overseen would be minting its own overseers. Work schedules sit with Tourism because a schedule decides who counts as late. Approvals sit with Tourism because a hand-written attendance row needs a second signature from someone who did not ask for it — so no one person can both file a correction and approve it, and nobody can file one for themselves.

Sign out from the menu under your name whenever you leave a shared computer. Changing your password signs out every other session on your account.

A3 · The visitor's journey on a phone

This is the flow a panel will most likely ask to see end to end. Nothing here is installed from an app store: the visitor app is a website that installs itself from the browser.

A3.1 The sequence

#	Screen	What happens
1	Register or Sign in	The visitor scans the entrance poster's QR code, or types the address. They register with an email address and a password of their choosing, or sign in if they have visited before.
2	Visitor information	Name, age, sex, visitor type (local resident, domestic tourist, foreign visitor) and where they are from. A Baler resident picks their barangay. Someone joining a party already registered at the desk enters the party's six-letter code here.
3	Waiting for the desk	The screen says either <i>Hand your admission fee to the staff at the desk or Show your ID at the entrance desk to claim free entry</i> . No exhibit content exists on the phone yet. The screen unlocks by itself the moment staff act.
4	How would you like to explore?	Two modes. Follow the Storyline walks the curated narrative order and, after each scan, highlights the next stop on the map. Explore Freely allows any exhibit in any order. The choice can be changed later in Settings.
5	Home	The starting point: the scanner, the exhibit list, the map, and the visitor's own progress.
6	Scan	Either point the camera at the QR label on the display case, or point it at the object itself and let the app recognise it (section A10). If neither works, the exhibit code printed under the QR square can be typed instead.
7	Exhibit	Three tabs: Overview (the description, with the narration player), Fun Facts , and Gallery . Narration can start automatically on scan and plays at an adjustable speed. Where no recording exists for the chosen language, the phone reads the description aloud instead and says so.
8	Map	Both floors, with a pin per exhibit, the ones already found marked, and in storyline mode the path from the current stop to the next.
9	Profile	What the visitor has scanned, what they bookmarked, and which halls they have covered.
10	Feedback	The ARTA Client Satisfaction sheet. A visitor who was on a guided tour is additionally asked about their guide; nobody else sees that question.

A3.2 Settings the visitor controls

Setting	Effect
Content language	Which language the exhibit text and narration are served in: English, Filipino or Spanish
Explore mode	Switches between Storyline and Free at any time
Audio narration, play on scan, playback speed	Whether narration starts by itself, and how fast it reads
Background music	Ambient audio, off by default
Text size, dark mode, high contrast	Accessibility controls; text size scales the whole interface, not only body text

A3.3 What the visitor keeps when the signal drops

Inside thick museum walls a phone often loses its connection. The app keeps its own copy of the screens, pictures, audio and the exhibits the visitor has already opened, and serves that copy when the network fails outright, so a tour does not end with an empty screen.

What deliberately does **not** work without a connection: registering, signing in, recording a scan, submitting feedback, and checking in at the geofence. Each says so plainly. There is no queue that replays these later — a scan recorded an hour late would put visitors in the wrong hall on the evening's report.

The app needs a secure connection. The camera, the location check and the offline copy all require the address to begin with `https://`. Opened over a plain local network address the app still runs, but with typed exhibit codes only and no attendance.

A4 · The front desk

Front Desk in the left menu, on the counter tablet or computer. This screen replaces the paper logbook, prices admission, and is the only place a visitor becomes cleared to see the collection.

A4.1 Three ways a visitor gets in

1. **On their own phone.** They scan the entrance poster, create an account, and the app tells them to come to the desk for their fee or ID check. Staff type nothing.
2. **On the counter tablet.** The same app on the museum's own tablet, turned round for the visitor to fill in.
3. **Staff type it.** For a visitor with no phone: **Register visitor** on the Front Desk page — name, visitor type, where they are from, and for a Baler resident, their barangay.

Front Desk
Register arrivals and collect admission — this replaces the logbook

Generate Report Entrance poster Guided Tours

15 PEOPLE TODAY 5 ENTRIES LOGGED

One visitor
Someone arriving on their own, or without a phone

FIRST NAME * LAST NAME *

VISITOR TYPE *

Local Free check ID Tourist PHP 50.00 Foreign PHP 50.00

FROM (CITY OR COUNTRY) AGE

Baler, Quezon City, Japan...

Register visitor

A group arriving together
Four or five from out of town, a family, or a school tour — one entry, one payment

WHO IS SIGNING IN * HOW MANY *

Name of the person at the desk 4

GROUP TYPE * VISITOR TYPE *

Group of friends / co-travellers Tourist (other province)

HOW MANY ARE FROM BALER? (free — check their IDs) FROM

0 City or country

Collect PHP 200.00 — 4 paying of 4.

Register group

Staff check-in code
Staff scan this on arrival with their own phone

1. Open the admin panel on your phone and sign in
2. Tap: My Attendance
3. Point your camera at this code

Rotates every 50s location checked 42s Open full screen --

Registered today All records --

TIME	GROUP	CODE	JOINED	FROM BALER	FEE	PAYMENT
11:15 AM	Nestor Del Rosario's group Group: 5 people Baler	0 / 5	0 of 5	Correct	PHP 250.00	Unpaid Mark Paid
10:33 AM	Ramon Garcia's group Group: 5 people Talar	0 / 5	0 of 5	Correct	PHP 250.00	Paid

TIME	NAME	TYPE	HOW	PAYMENT
2:13 PM	Migs Cruz	Tourist	App	Unpaid
12:29 PM	Migs Aquino	Tourist	App	Unpaid
10:39 AM	Grace Reyes	Local	App	Free
9:57 AM	Cristina Bautista	Tourist	Desk	Paid
9:46 AM	Migs Garcia	Tourist	App	Paid

Administrator Museum Staff

Password Alerts Log out

Figure A4.1 — The Front Desk screen: registration, today's arrivals, and the group register with their payment state.

A4.2 Clearing a visitor

Nobody sees exhibit content until the desk clears them. Until then their screen shows the waiting message, and it unlocks the moment staff act.

Visitor type	What staff do	Result
Domestic tourist or foreign visitor	Collect the fee, then Mark Paid	Admission recorded at the current fee, with the staff member's name and the time. Their phone unlocks by itself.
Local resident of Baler	Sight a valid ID showing a Baler address, then Verify ID	Free admission, recorded with who verified it. If the ID is from elsewhere, they register as a tourist instead.

Both actions are available on the Front Desk page's *Registered today* list and in **Records**.

A4.3 Groups: schools, families, tour agencies

A party is one record and one payment, not thirty registrations.

1. **Register group**: who is signing in, the group type, how many people, where they are from, and **how many of them are Baler residents** — those are free, so check their IDs.
2. The screen shows the total fee and a **six-letter code**. Read the code to the group leader. Anyone in the party who wants the app on their own phone enters that code when they register, and is counted inside the group instead of owing a second fee.
3. Collect the fee and **Mark Paid**. Every member's phone unlocks at once.

A group code cannot be abused. It works only on the day it was issued, and only for as many people as were counted. A code passed around outside cannot admit a sixth person to a party of five. Codes contain no letter O or I, so they cannot be misread as zero or one.

A4.4 Refunds and corrections

Situation	What to do
A Baler resident was charged inside a group, discovered after registering	On the group's row, change the <i>from Baler</i> count and click Correct . The group is re-priced. If it had already paid, the difference is recorded as a refund with the staff member's name — hand the cash back and the books match. Same-day only.
An individual visitor paid but should not have	There is no undo for an individual payment in the panel, on purpose. Note it in the logbook remarks and tell the Tourism office the same day; it is corrected in the records.
Marked paid by mistake, no money taken	The same: tell the Tourism office the same day.

A4.5 End of day

Records lists everyone from today with their payment state. **Records** → **Logbook** prints the day, or exports it as a spreadsheet or PDF, for the cash count (section A13).

A5 · Exhibits: add, edit, archive

Exhibits in the left menu. Every item a visitor can scan or find on the map is an exhibit.

A5.1 Adding an exhibit

The screenshot shows the 'Add Exhibit' form in the Museo de Baler system. The form is a modal window with the following fields and options:

- EXHIBIT CODE:** A text input field with the example 'e.g. EXH-009'.
- CATEGORY:** A dropdown menu with the option 'None'.
- NAME:** A text input field for the exhibit name.
- HALL:** A dropdown menu with the option 'No hall yet'.
- AUTHORS:** A text input field with the example 'e.g. Dr. Juan Dela Cruz'.
- DESCRIPTION:** A text input field for the exhibit description.
- FUN FACTS (one per line):** A text input field with the instruction 'Enter each fun fact on a new line...'.
- TRANSLATIONS & AUDIO GUIDE:** A section with a 'WRITTEN IN' dropdown menu set to 'English', a 'Generate with AI' button, and a 'Narrate all' button. Below these are instructions: 'Read each translation, fix anything, listen to the audio — then save the exhibit.' and 'Add language: English Filipino Spanish'.
- LANGUAGES:** A section with a 'STORYLINE ORDER' dropdown menu.

Figure A5.1 — The Add Exhibit form. Translations and narration are drafted on the same page and saved together with it.

Field	What it is for
Exhibit Code	Printed on the QR label and typed by visitors when the camera fails. Short, no spaces, unique, for example EXH-012 . It cannot be changed after labels are printed without reprinting them.
Name and Description	What the visitor reads. Write these first; the translations and narration are made from them.
Fun Facts	One per line. Leave it empty rather than padding it — the app hides the tab when there are none.
Category, Hall	The hall list comes from Museum Info (section A9).
Authors	Who curated or wrote it; shown to visitors.
Languages	Which languages this exhibit is offered in.
Storyline Order	The number of this stop on the guided path (1, 2, 3 ...). Leave blank for an exhibit that is not on the path.
Image	JPG, PNG, GIF or WebP up to 10 MB. This is the picture visitors see and the one the camera compares against.

If the form comes back with a red message, fix the field it names and **choose the picture again** — browsers do not keep a selected file when a form is sent back.

A5.2 Editing an exhibit

Museo de Baler
ADMIN PANEL

Edit Exhibit
Siege of Baler

Exhibit Details

EXHIBIT CODE: EXH-001 CATEGORY: History

NAME: Siege of Baler

HALL: Hall A — History & Religion · G

AUTHORS: Museo de Baler Curatorial Team

DESCRIPTION:
From July 1, 1898 to June 2, 1899, a garrison of 54 Spanish soldiers under Captain Enrique de las Morenas fortified themselves inside the San Luis Obispo de Tolosa Church in Baler. Filipino revolutionary forces laid siege for 337 days — one of the longest last stands of the Spanish colonial era. Remarkably, the soldiers held out even after Spain had already ceded the Philippines to the United States through the Treaty of Paris on December 10, 1898. The siege ended when Lieutenant Saturnino Martin Cerezo finally surrendered, unaware the war had long been over. This exhibit features a diorama of the siege, period weapons, and historical documents.

FUN FACTS (ONE PER LINE)
The siege lasted 337 days — the longest holdout of the Spanish colonial period in the Philippines.
The Spanish soldiers were unaware that Spain had already surrendered the Philippines to the US months earlier.
Captain Enrique de las Morenas died of illness during the siege; Lieutenant Martin Cerezo took command.
Only 33 of the original 54 soldiers survived to surrender on June 2, 1899. The event is commemorated annually on Philippine-Spanish Friendship Day.

LANGUAGES: Filipino, English STORYLINE ORDER: 1

IMAGE:
Choose File | No file chosen

Translations & Audio + Add

English (en)
Siege of Baler
0:00 / 0:39

Filipino (fil)
Pagkubkob ng Baler
0:00 / 0:52

Camera Recognition
0 training photo(s) · needs 30+ to be recognised well Manage photos

Gallery Images

ADD IMAGES
Choose Files | No file chosen

CAPTION (OPTIONAL)
e.g. Front view

Upload

Cancel Save Changes

Administrator
Museum Staff

Password Alerts Log out

Figure A5.2 — The edit page also carries Translations & Audio, Camera Recognition photos, and the gallery.

Open the exhibit, click **Edit**, change what you need, and **Save Changes**. The same page holds **Translations & Audio** (section A6), **Camera Recognition** photos (section A10), and **Gallery Images** — extra pictures with captions shown in the app's gallery tab. Changes reach visitors' phones the next time the app refreshes its list, usually within a minute.

A5.3 Archive and restore

Archiving hides an exhibit from visitors and from the map without deleting anything — its scans, photos and translations stay. Open the exhibit and click **Archive**; it disappears from the app at once and from the QR sheet. To bring it back, set the filter on the Exhibits page to **Archived**, open it, and click **Restore**.

Use archive for a piece on loan or in conservation. **There is no button that deletes an exhibit outright, on purpose** — its scan history and printed QR labels outlive it.

A6 · Translations and the audio guide

Each exhibit can carry a title, description and spoken narration in several languages; visitors pick their language in the app. The panel supports **English**, **Filipino** and **Spanish**. There are two ways to fill these in, and they can be mixed.

A6.1 Method A: let the AI draft them

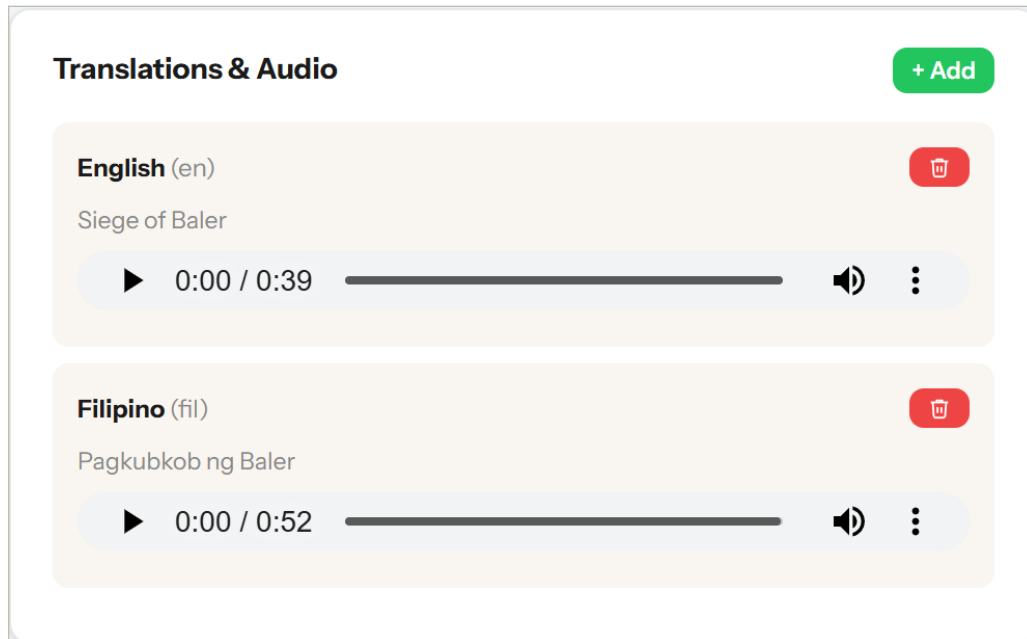


Figure A6.1 — The Translations & Audio Guide section. Drafts are marked *AI draft — review* until a curator has read them.

1. Write the **Name**, **Description** and **Fun Facts** first — that is what gets translated.
2. Click **Translate**. A card appears per language, marked *AI draft — review*. **Read each one**. The AI is good but it is not a curator: check names, dates and anything with a local meaning. Edit the text in the card directly.
3. Click **Narrate all**, or **Narrate** on one card. This takes a few seconds per language. A player appears — **listen before saving**.
4. If you edit a card's text after narrating it, the card warns *Text changed since this was narrated*. Narrate again, or the voice will read the old text.
5. **Nothing is stored until you save the exhibit**. Closing the form discards the drafts.

Limits, so nobody is surprised

- **There is a budget:** 20 requests a minute and 300 a day per staff account. Past that the button says to wait; the daily count resets the next day. Narrating three languages for one exhibit is three requests.
- **It needs the internet and a Google AI key** configured on the server. If the button says *AI generation is not set up*, that key is missing.
- **If it says the text was declined**, the wording tripped Google's content filter. Reword and try again; the draft is not saved.
- **The museum works without it**. With no key at all, every other feature is unaffected and translations are typed by hand.

A6.2 Method B: type a translation and upload a recording

On the edit page, **Translations & Audio** → **+ Add**:

1. **Code** (en, **fil**, es) and **Label** — the name visitors see.
2. **Title** and **Description** in that language.
3. **Audio** (MP3 or WAV) — a recording by a staff member or a hired narrator. Keep it to a few minutes; the app streams it, and a visitor on weak Wi-Fi waits for the whole file.
4. **Save**. To replace a recording, delete the translation and add it again.

The two methods mix freely: AI text with a human recording, or a human translation with AI narration.

A7 · QR codes and the entrance poster

Every active exhibit has a QR code that opens it in the app. Visitors scan it with the app's scanner, or with their phone's own camera, which opens the app.

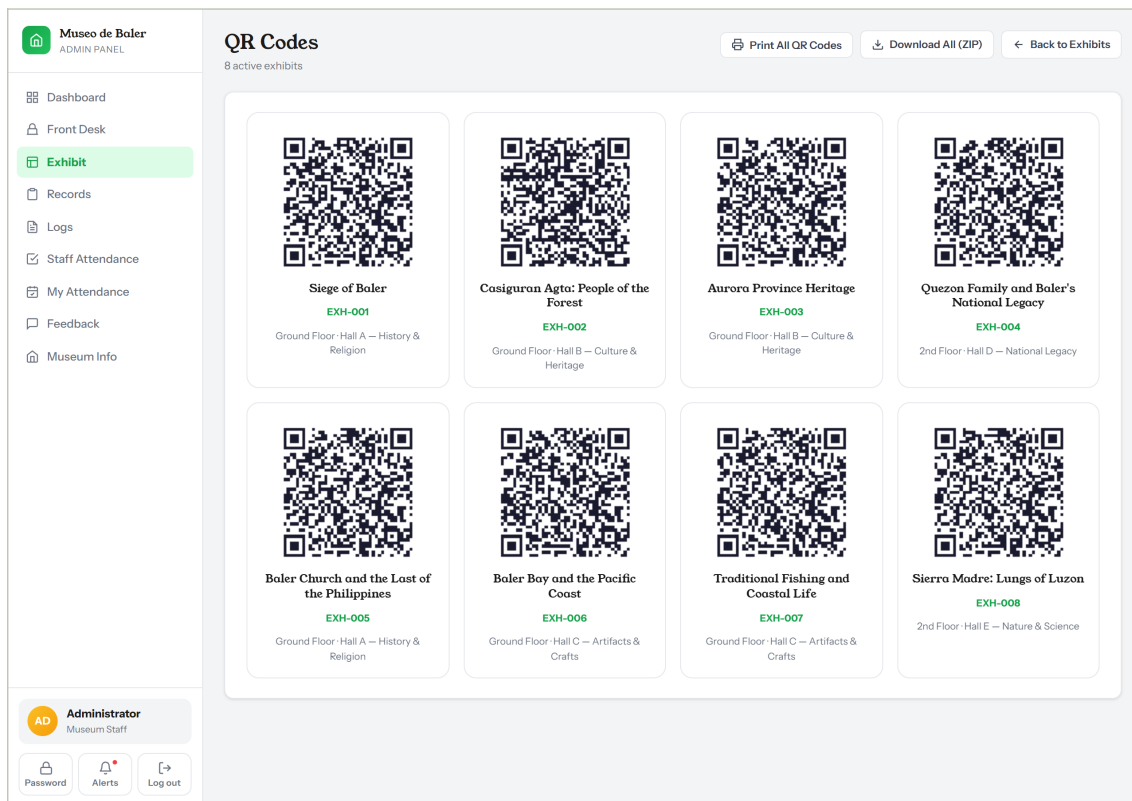


Figure A7.1 — The QR Codes page: one label per active exhibit, printable as a sheet or downloadable individually.

Option	What you get
Download SVG (one exhibit)	A single label as a vector file. Print it at any size; it stays sharp.
Print All QR Codes	A sheet of every active exhibit, from the browser's print dialog. Plain paper or sticker sheets.
Download All (ZIP)	Every label as a separate vector file in one download, for a print shop.

Each label carries the exhibit's code in plain text under the QR square, so a visitor whose camera will not focus can type it instead.

Where to put them

At reading height, in the light, and not behind glass that reflects. A label under a spotlight glare will not scan; the app then tells the visitor to type the code.

The entrance poster

Front Desk → Poster is a different QR code: it opens the app's sign-in page, for visitors who arrive without having installed anything. Print it and stand it at the door. This is how self-registration happens when there is no counter tablet.

A8 · The museum map and the storyline

Map in the menu. Visitors see the same map in the app, with the exhibits they have found marked and, in storyline mode, the path from stop to stop.

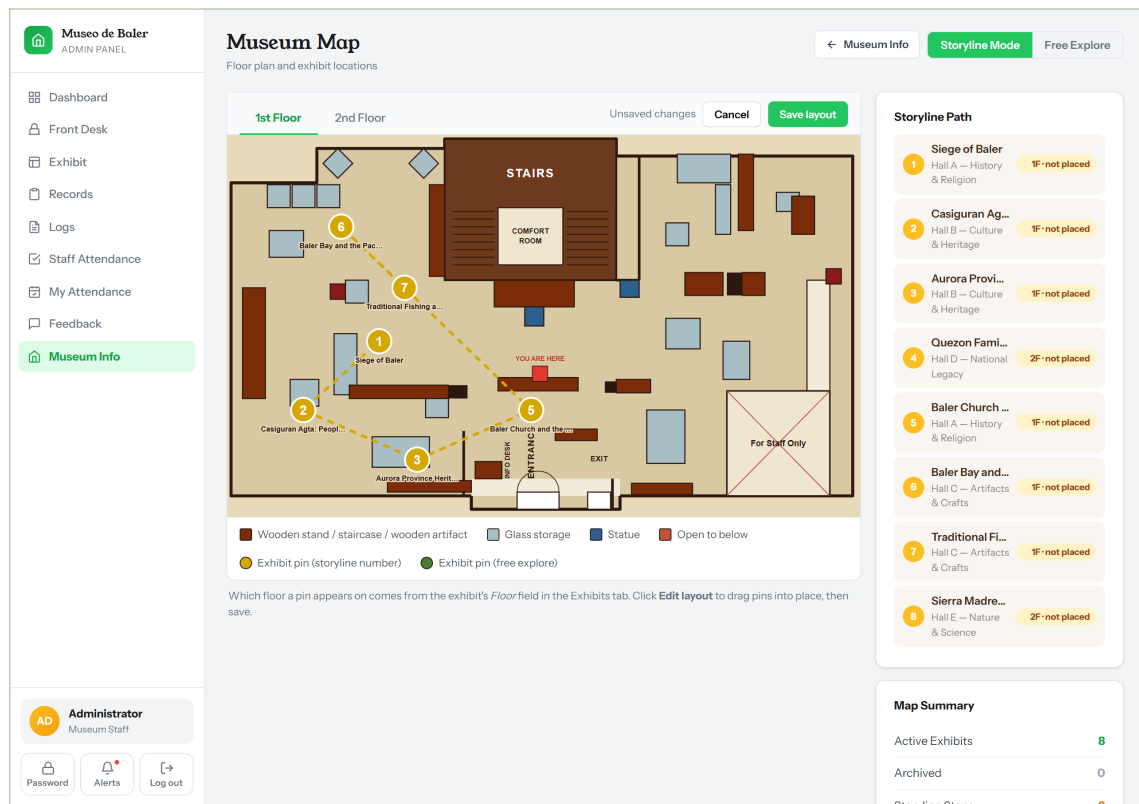


Figure A8.1 — The map editor. Pins are dragged onto the floor plan and saved; the storyline path is listed alongside.

1. Switch between **1st Floor** and **2nd Floor** at the top.
2. Click **Edit layout**.
3. **Drag** each exhibit pin to where the piece actually stands. An exhibit that has never been placed appears in a default spot; put it right.
4. Click **Save layout**. Nothing moves for visitors until you do.
5. The **Storyline Path** panel lists the stops in order and warns when an exhibit is missing a storyline number — set that in the exhibit's own form (section A5).

The floor plan drawing itself — the walls, rooms and stairs — is part of the software, not something staff redraw here. Exhibit *pins* are data and staff own them. If a wall moves, the maintainer edits the drawing; see Part B, section B15.

A9 · Museum info: hours, fee, halls, the geofence

Museum Info in the menu. This is what visitors read on the app's About screen and the numbers the desk works from. Click **Edit**, change what you need, then **Done**.

Figure A9.1 — Museum Info: the About screen text, the admission fee, the halls, and the geofence that both the visitor check-in and staff clock-in read.

A9.1 The fields

Field	Notes
Museum Name, Tagline, Our Story, Second Paragraph	The app's About screen.
Address, Opening Hours, Closed On, Phone, Email	Also the About screen. Keep <i>Closed On</i> current — the app shows it and visitors plan by it.
Admission Fee (PHP, per visitor)	The flat fee for tourists and foreign visitors; Baler residents are free. Change it here and the desk, the app and the reports all use the new amount from the next registration onward. It does not change what people have already paid. The sentence <i>shown to visitors as</i> is written from the number, so it can never disagree with what the desk collects.
Museum Halls	The halls exhibits are assigned to. Add one with a name, floor and description. Delete one only when no exhibit uses it.

A9.2 The geofence

The app records a visitor as being in the museum when their phone is inside a circle drawn around the building, and staff can only clock in from inside that same circle.

Field	Notes
Latitude, Longitude	The centre of the circle. Set it once, from a phone standing at the entrance: on the phone, open My Attendance → Use my current location .
Geofence Radius (metres)	150 m covers the building and its forecourt. Larger catches phones out on the street; smaller loses people at the back of the building.

Do not set the pin from a desktop computer. A desktop guesses its position from Wi-Fi, and during testing that put the pin 1.8 km from the door — after which nobody could clock in and no visitor was counted as present. This is why the panel offers a phone button for it.

A10 · Camera recognition

Besides the QR labels, the app can identify an exhibit through the phone camera. It works only after staff have taught it, and it fails softly: with no training the app falls back to a weaker server-side match, and **the QR codes always work regardless**.

EXHIBIT	CODE	TRAINING PHOTOS	IN CURRENT MODEL	
Siege of Baler Hall A — History & Religion	EXH-001	0 • too few	• yes	Manage photos
Casiguran Agta: People of the Forest Hall B — Culture & Heritage	EXH-002	0 • too few	• yes	Manage photos
Aurora Province Heritage Hall B — Culture & Heritage	EXH-003	0 • too few	• yes	Manage photos
Quezon Family and Baler's National Legacy Hall D — National Legacy	EXH-004	0 • too few	• yes	Manage photos
Baler Church and the Last of the Philippines Hall A — History & Religion	EXH-005	0 • too few	• yes	Manage photos
Baler Bay and the Pacific Coast Hall C — Artifacts & Crafts	EXH-006	0 • too few	• yes	Manage photos
Traditional Fishing and Coastal Life Hall C — Artifacts & Crafts	EXH-007	0 • too few	• yes	Manage photos

Figure A10.1 — The Recognition page, showing how many training photos each exhibit has and the button that trains the model.

1. **Recognition** in the menu. Each exhibit shows its photo count: it needs **at least 20, and 30 or more to work well**.
2. **Manage photos** on an exhibit. Do this **on a phone** — this page is deliberately allowed on a phone because that is where the camera is. Stand where a visitor would stand and take photos from several angles, distances and times of day.
3. **Background photos**: 20–30 pictures of walls, floors, corridors and other objects — things that are *not* an exhibit — so the camera learns to say “nothing here” instead of guessing.
4. Back on a computer, click **Train the model**. The training runs in the browser and takes a minute or two; **do not close the tab**. Every phone picks up the new model the next time the app refreshes.

Retrain after adding an exhibit, moving one to a different-looking spot, or changing its lighting.

A11 · Guided tours and visitor feedback

A11.1 Tours

Most visits are unguided, so this screen is deliberately small. **Start tour** — pick the guide and who the tour is for (one visitor or a group). **End tour** when it is done. A visitor who was guided is asked in the app's feedback sheet how their guide was; nobody else is shown that question.

A11.2 Feedback and the ARTA survey

Feedback lists what visitors said, together with the ARTA Client Satisfaction Measurement survey their answers feed. **Reports → Feedback** produces the CSM figures for a date range and the file the LGU filing needs (section A13). The Tourism office owns the questions themselves (section A12.4).

A12 · The Tourism office's screens

These four areas belong to the Tourism Head role and are not visible to museum staff at all.

A12.1 Staff accounts

Staff → Add Staff: full name, email address, role. **The system generates the password and shows it once** — copy it and hand it over in person. The staff member must change it at first sign-in.

- **Reset password** on an existing account does the same for someone who has forgotten theirs. It is a separate action from editing the account, so it gets its own audit entry and cannot happen by accident while someone is correcting a spelling.
- **Toggle** disables an account without deleting it — for someone who has left. Their past actions stay in the records.

A12.2 Staff attendance

Museum staff clock in by scanning the code on the staff-room tablet (**Attendance → Kiosk**, left running) with their own phone, from inside the geofence. **The kiosk code rotates every 60 seconds**, so a photograph of it is worthless.

- **Staff Attendance** shows the sheet for a period.
- **DTR** prints or exports the month for one person (section A13).
- **Schedule**, per staff member, is what decides who counts as late.

A12.3 Attendance corrections

If someone forgot to clock in, **museum staff file a correction** with the reason, and **the Tourism office approves or rejects it**. Nobody can do both halves, and nobody can file one for themselves. Both roles can read the list; only Tourism can act on it.

A12.4 The visitor survey

Survey edits the question bank the app shows visitors. The questions are the ARTA Client Satisfaction Measurement form the office files with the LGU, so the office that answers for the numbers owns the questions. The standard ARTA set is locked; extra questions can be added or retired. **A retired question keeps its answers in past reports** — retiring it changes what new visitors are asked, never what old reports say.

A13 · Reports and exports

There is no single reports hub page. Each report is reached from the section it belongs to — the logbook from Records, the DTR from Staff Attendance — because that is where the person who needs it already is.

A13.1 The six reports and their formats

Every report can be previewed on screen before committing to a download, so nobody spends a download working out whether the date range was right.

Report	What it contains	CSV	XLSX	DOCX	PDF
Logbook	One day's arrivals, individuals and groups, with payment state — the cash count	Yes	Yes	—	Yes
DTR	One staff member's daily time record for a month	Yes	Yes	—	Yes
Visitors	Visitor counts and composition over a date range	Yes	Yes	—	Yes
Exhibits	The collection and how often each piece was scanned	Yes	—	Yes	Yes
Feedback	Visitor feedback and the ARTA CSM figures	Yes	—	Yes	Yes
Audit trail (Tourism only)	Every change made in the panel, by whom and when	Yes	—	—	Yes

Why the format list differs per report. The formats offered are the ones each report is actually wanted in. A DTR and a logbook are columns of figures that an office re-totals, so they offer a spreadsheet. The exhibit and feedback reports are narrative documents that get pasted into a larger submission, so they offer Word. Every report offers CSV for machines and PDF for filing. An unsupported combination is refused by the system rather than producing an empty file.

A13.2 The audit trail is Tourism-only

The audit trail records what everyone did, **including the museum staff it is kept on**. It is therefore exported only by the Tourism office, and is deliberately excluded from the shared export route that serves the other five reports.

A14 · Troubleshooting

A14.1 When a visitor asks for help

What their phone says, what it means, and what to do.

The visitor's screen says	What is going on	Do this
<i>Hand your admission fee to the staff at the desk...</i>	They registered and are waiting for you.	Collect the fee, Mark Paid . Their phone unlocks by itself.
<i>Show your ID at the entrance desk to claim free entry.</i>	A Baler resident waiting for an ID check.	Check the ID, Verify ID .
<i>Your group's admission has not been recorded yet.</i>	They joined a group that has not paid.	Find the group leader; Mark Paid on the group.
<i>That group code was not found for today.</i>	Wrong code, or the group was registered on another day.	Read the code from the group's row on the Front Desk page. Codes contain no O or I.
<i>Everyone in that group has already joined.</i>	The party is bigger than the headcount entered.	Correct the headcount on the group's row, then they can join.
<i>That email is already registered. Tap Sign In instead.</i>	They have visited before.	They sign in with the password they made then. There is no password reset for visitors; they may register with a different address.
<i>That email address does not look real.</i>	The part after the @ is misspelled, for example gmial.com.	Ask them to check it.
<i>Incorrect email or password. / Too many attempts.</i>	Wrong password; after six tries a one-minute pause.	Wait a minute and try again.
<i>Can't reach the museum right now.</i>	Their phone has no signal or Wi-Fi.	Point them to the museum Wi-Fi. Inside the halls the app keeps working from what it already loaded; sending feedback needs a connection.
<i>Exhibit not found: ...</i>	They typed a code that is not an active exhibit.	Check the label; the code is printed under the QR square.
<i>The camera needs a secure (https) connection.</i>	They opened the app through the wrong address.	Give them the address from the poster — it starts with https.
<i>No back camera was found / The camera is being used by another app.</i>	A problem with their phone, not the museum.	Close other camera apps, or type the exhibit code.
<i>No exhibit recognized — try moving closer</i>	Camera recognition did not match.	Scan the QR label instead. If this happens often for one exhibit, it needs more training photos (section A10).
<i>Narration file unavailable — reading the description instead</i>	No recording for that language; the phone reads the text aloud.	Nothing to do. Add narration later (section A6).
<i>Your session has expired. Please sign in again.</i>	Visitor sessions last 24 hours.	They sign in again; nothing is lost.

If a visitor asks whether the app tracks them: it records when their phone enters and leaves the museum's circle, which exhibits they opened, and what they wrote in feedback. Nothing outside the museum grounds. Nothing is sold or shared with anyone. Part B, section B16 sets out exactly what is collected and why, for anyone who wants the full answer.

A14.2 When the panel shows an error

Message	Meaning	Do this
<i>Page Expired / Your session expired. Reload the page and scan again.</i>	A page was left open too long.	Reload and redo the last action. Nothing was saved twice.
<i>Too many requests on the AI buttons</i>	The per-minute or daily AI budget is used up.	Wait; the daily count resets the next day.
<i>AI generation is not set up...</i>	The Google AI key is missing on the server.	Tell the maintainer. Everything else works.
<i>Gemini refused the request / declined this text</i>	Google's service or content filter.	Try again later, or reword the text.
<i>Could not create ZIP archive</i>	The server is missing its ZIP support.	Use Print All or single Download SVG ; tell the maintainer (Part B, B15).
<i>Only today's groups can be corrected</i>	The group is from a past day.	Corrections are same-day. Tell the Tourism office.
<i>You must be at the museum to record attendance</i>	Clock-in from outside the geofence, or a vague GPS fix.	Step inside, or nearer a window, and scan again.
<i>That is not a Museo de Baler attendance code, or the code expired</i>	The kiosk code rotates every minute.	Scan the code on the kiosk screen now, not a photograph of it.
<i>Something went wrong on our side</i>	A real fault. The maintainer is emailed automatically.	Note what you were doing and the time, then try once more.
The site does not load at all	The server or the internet connection is down.	Take names on paper and enter them under Front Desk later. Visitors already inside the app keep working from what their phones hold.

A15 · Routine maintenance

A15.1 Weekly, whoever opens the museum

- Glance at the dashboard. Yesterday's visitor count should be there. **If the last few days show zero while people came**, the geofence pin or the app's address has changed — check section A9.
- Confirm the nightly backup ran: there should be two new files carrying yesterday's date in the backup folder. If there are none for two days the system emails the alert address, so make sure someone reads that inbox.

A15.2 Quarterly, the Tourism office with the maintainer

- **A restore drill.** Follow the restore procedure end to end on a scratch database and record the date. A backup nobody has ever restored is a hope, not a backup.
- **Review who holds what.** Is every account, key and credential still held by someone who works here? The access register in the repository has a table for this.
- **Reprint any QR labels that have faded** or been damaged.

The single most important item in this manual. The nightly backup is encrypted with a key kept in the server's configuration. **If that key is lost, every backup ever made with it is permanently unreadable — by anyone, including the people who wrote this system.** The key must exist in two places that cannot fail together: on the server, and somewhere that is not the server (printed and sealed in the Tourism office safe, or in the office password manager). A copy on the same machine or the same cloud account as the backups does not count. See Part B, section B13.

P A R T B

Technical Reference

The stack, the architecture, the request pipeline, the data model, the API surface, the security controls, and the procedures for installing, deploying, backing up and verifying the system. Written for an IT evaluator or the developer who inherits the code.

B1 · Stack and inventory

B1.1 Platform

Component	Version and notes
Language	PHP 8.3 in production; <code>composer.json</code> requires <code>^8.2</code> . Extensions: <code>mbstring</code> <code>gd</code> <code>sodium</code> <code>pdo_mysql</code> <code>zip</code> <code>xml</code> <code>curl</code>
Framework	Laravel 12 (<code>laravel/framework ^12.0</code>), server-rendered Blade for the panel
Database	MySQL 8. Both connections pin MySQL's session time zone to <i>Asia/Manila</i>
Visitor app	Plain HTML, CSS and JavaScript — no build step and no framework . A service worker provides offline fallback; the app installs from the browser as a PWA
Panel styling	Vite, one stylesheet (<code>npm run build</code>)
Image recognition	A Teachable Machine image classifier, roughly 3 MB of static files, running in the visitor's browser
Test runner	PHPUnit 11 on in-memory SQLite; <code>node --test</code> for the service worker
Node	20 or newer (24 on the development machine)

Production dependencies, and what each is for

Package	Why it is here
<code>laravel/framework ^12.0</code>	The application framework
<code>chillerlan/php-qrcode ^6.0</code>	QR labels for the display cases, as SVG, and the rotating staff-attendance kiosk code
<code>mpdf/mpdf ^8.3</code>	PDF reports. Chosen over lighter converters because its HTML header support repeats the museum letterhead on page two onward — a letterhead on only the first page of a twelve-page DTR is not a letterhead
<code>phpoffice/phpspreadsheet ^5.10</code>	XLSX exports (logbook, DTR, visitors)
<code>phpoffice/phpword ^1.4</code>	DOCX exports (exhibits, feedback)
<code>laravel/tinker</code>	REPL for maintenance

There is **no** queue worker, websocket server, Redis instance or search engine to keep alive. The database is the cache, session and queue driver. This is a deliberate operational choice for a municipal museum with no full-time systems administrator.

B1.2 Code inventory

Counted from the working tree at the time of writing.

Item	Count
Registered routes (total)	119
of which the visitor API (<code>/api/v1</code>)	16
of which the panel and framework endpoints	103
Panel controllers	18
API controllers	10

Item	Count
Middleware classes	11
Eloquent models	23
Migrations	51
Database tables (23 domain + 8 framework)	31
Artisan console commands (project-specific)	7
PHPUnit tests / assertions, all passing	406 / 4,902
Service-worker tests, all passing	15

B1.3 What is authored and what is generated

A frequent question, because the folder on disk is far larger than the project. Everything not checked into version control is either generated by a tool or uploaded by a user, and all of it is listed in `.gitignore`.

On disk	Size	In the repo?	Where it comes from
<code>app/ routes/ resources/ config/ database/ tests/ docs/ public/visitor/</code>	~4 MB	Yes	Written by the team — this is the project
<code>vendor/</code>	170 MB	No	<code>composer install</code> reads <code>composer.lock</code>
<code>node_modules/</code>	36 MB	No	<code>npm ci</code> reads <code>package-lock.json</code>
<code>public/images/ public/audio/</code>	103 MB	No	Uploaded exhibit photos, training photos, narration
<code>storage/app/backups/</code>	~108 MB	No	<code>php artisan db:backup</code> , pruned automatically
<code>storage/logs/ storage/framework/</code>	~11 MB	No	Runtime logs and compiled Blade cache
<code>public/build/</code>	~1 MB	No	<code>npm run build</code>

The two lock files are the point: `composer.lock` and `package-lock.json` are committed and pin every dependency to an exact version. That is why `vendor/` need not be — any machine rebuilds it identically, and continuous integration does exactly that on every push before running the suite.

B2 · Architecture

B2.1 The three faces



The separation is strict and tested. **The visitor app never renders a Blade view and never holds a session cookie; the panel never serves JSON to a phone.** The `DesktopOnly` middleware enforces the second half and a feature test pins it.

B2.2 External dependencies, both optional

Service	Used for	What happens without it
Google Gemini	Drafting exhibit translations and generating narration audio	The two buttons say the feature is not configured. Everything else works; translations are typed and recordings uploaded by hand. Only exhibit text is ever sent — never a visitor record.
SMTP	Alerting a maintainer	Unhandled errors and failed or stale backups go unreported. The system keeps running, but blind.

B3 · The request pipeline

Every request passes the same ordered gates. The order is the design: input is sanitised before a controller sees it, and the audit log records an act only after the role check has allowed it.

B3.1 The panel (session-authenticated)

```
Request
|
+-- [Laravel web group] cookies, encrypted session, CSRF, route bindings
+-- SanitizeInput ..... strips HTML tags and control characters from
|                          every string field, before validation
+-- AuthenticateSession .... binds the session to the password hash it was
|                          opened with, so a password change really does
|                          end other sessions
+-- SecurityHeaders ..... CSP, X-Frame-Options: DENY, nosniff,
|                          Referrer-Policy, Permissions-Policy
+-- AuditLog ..... records every POST/PUT/PATCH/DELETE
+-- BackupWatchdog ..... warns when the newest dump is stale
|
+-- [per route group]
|   auth ..... web guard, App\Models\Staff
|   throttle:panel ..... 300 requests/minute/account
|   password.rotate ..... an unchanged issued password goes no further
|   |                      than the change-password screen
|   desktop ..... the panel is a desktop tool; a phone session
|   |              is narrowed to clock-in and password change
|   role:... ..... Administrator, TourismHead, or both
|
v
Controller --> Service / Support class --> Response
```

B3.2 The visitor API

```
Request
|
+-- ApiResponseHeaders ..... PREPENDED, not appended
|                          nosniff, Cross-Origin-Resource-Policy,
|                          Referrer-Policy, Cache-Control: no-store,
|                          X-Powered-By removed
+-- HandleCors ..... origins from API_ALLOWED_ORIGINS only
+-- throttle:visitor-api ... 60 requests/minute/address
+-- SanitizeInput ..... the same sanitiser as the panel, on JSON bodies
|
+-- [per route tier]
|   (public) ..... GET museum, GET notifications
|   visitor.auth ..... 64-hex bearer token resolves to a Visitor
|   visitor.cleared ..... AND the desk has taken the fee or sighted an ID
|
v
Controller (Form Request validation) --> JSON
```

Why ApiResponseHeaders is prepended rather than appended. Appended, it never ran on the one response a probe is most likely to collect: a 429 Too Many Requests came back with no nosniff, no cross-origin policy, the wrong Referrer-Policy and PHP's X-Powered-By still attached, because the rate limiter had already ended the request before the header middleware was reached. Laravel renders an exception into a response *inside* the pipeline, so an outer middleware still gets to decorate it. This is the kind of

detail worth asking about in review, and it is the reason for the ordering.

B4 · Authentication and authorisation

B4.1 Two populations, two guards, two tables

	Staff	Visitors
Guard	web — session cookie	visitor — bearer token
Table / model	staff / App\Models\Staff	visitors / App\Models\Visitor
Credential storage	bcrypt, 12 rounds	bcrypt, 12 rounds; token stored only as a SHA-256 hash
Lifetime	120-minute session, ends when the browser closes; encrypted, HttpOnly, SameSite=Strict, secure over TLS. “Remember me” is deliberately not offered	Token valid 24 hours
Password policy	12+ characters with mixed case, a digit and a symbol; may not contain the holder's name or the museum's; issued by Tourism and must be changed at first sign-in	Length plus a common-password blocklist, deliberately <i>without</i> character-class rules, following NIST SP 800-63B
Throttle	5 failed attempts per address per minute	6 per account per minute; 30 per address per 5 minutes
Reset	Tourism issues a new one; it must be changed on use	None by design — no mail is sent to visitors and there is no identity to verify against. They may register again with another address

Why staff and not Laravel's default users table. Museum employees are not generic users: they carry a role, a schedule, a daily time record and a password lifecycle. Two populations, two tables, two guards — so a visitor token can never yield a staff session, and the boundary is a structural fact rather than a check someone has to remember to write.

B4.2 Role enforcement

Two roles, `Administrator` and `TourismHead`, enforced server-side on every route by `EnsureRole` and grouped in `routes/web.php`. A denied request is a 403 and a line in the security log. Part A, section A2.1 has the full permission matrix.

Two separations of duty are enforced in code, not in policy:

- **Nobody can both file and approve an attendance correction**, and nobody can file one for themselves. Museum staff file; Tourism reviews.
- **Museum staff cannot create staff accounts.** The tier being overseen does not mint its overseers.

How to demonstrate it in review

- Sign in as museum staff and type `/staff` into the address bar — a 403 page.
- Sign in as Tourism and type `/desk` — a 403 page.
- `tests/Feature/RoleAccessTest.php` pins both directions.

B4.3 The admission gate

This is the system's central authorisation rule, and it is worth stating precisely:

A token gets a visitor as far as the desk.
Only a collected fee, or a sighted residency ID, gets them past it.

Exhibit content sits behind `EnsureVisitorCleared` in addition to `AuthenticateVisitor`. The check runs on the server on every request. A `visitor_id` supplied in a request body is **ignored unless it agrees with the token**, so no visitor can read or write another visitor's record, and every write — attendance, scans, feedback — is attributed to the visitor the token resolves to.

The app's “waiting for the desk” screen is a courtesy rendering of `GET /api/v1/visitors/me`. Editing the phone's local storage changes the screen, not the answer.

B5 · The visitor API

All 16 endpoints sit under `/api/v1` and answer JSON. Anything under `/api` is JSON regardless of the `Accept` header: a validation failure is a `422` with the errors, never a redirect to a login page.

B5.1 Endpoint reference

Verb	Path	Tier	Purpose and notes
GET	<code>museum</code>	public	The About screen: hours, fee, address, story, halls
GET	<code>notifications</code>	public	The notice board
POST	<code>visitors</code>	public, throttled	Register. 10 per minute per address
POST	<code>visitors/login</code>	public, throttled	Sign in. 6/min per account, 30 per 5 min per address
POST	<code>visitors/logout</code>	public	Discards the token
POST	<code>attendance</code>	token read if present	Geofence check-in
PATCH	<code>attendance/{id}</code>	token read if present	Check-out
GET	<code>media/{path}</code>	signed URL	Protected exhibit images and audio. See B5.2
GET	<code>visitors/me</code>	visitor.auth	The visitor's own standing with the desk — what the waiting screen polls
POST	<code>visitors/me/group</code>	visitor.auth	Join a party with its six-letter code
GET	<code>exhibits</code>	visitor.cleared	The collection. <code>private</code> , <code>no-cache</code> with an ETag, so an unchanged museum is a <code>304</code> with no body
GET	<code>exhibits/{code}</code>	visitor.cleared	One exhibit, with translations and signed media URLs
POST	<code>scans</code>	visitor.cleared	Record that this visitor opened this exhibit
POST	<code>recognition</code>	visitor.cleared , throttled	Resolve a predicted class to an exhibit. 120/min for camera sampling
GET	<code>survey</code>	visitor.cleared	The ARTA question bank as currently published
POST	<code>feedback</code>	visitor.cleared	Submit the survey

B5.2 Media is served through short-lived signed URLs

Exhibit images and narration are not public files. The cleared exhibit endpoints mint temporary signed URLs, and `GET media/{path}` validates the signature. The reason is mechanical: an HTML `<img>` or `<audio>` element cannot attach a bearer header, so the signature *is* the authorisation proof for that short-lived request.

The route uses `signed:relative`, not `signed`. The absolute check rebuilds the scheme and host before hashing, and therefore rejected every URL this application signed itself, because the URLs are minted relative. **The signer and the verifier have to agree about which form was signed.** On nginx, direct access to `/images/exhibits/` and `/audio/` must be denied before the generic static-file rule, or the web server would serve the same files without ever consulting the gate.

B5.3 Design note on API shapes

This API replaced a set of raw PHP scripts that talked to the same database through `mysqli` alongside the Laravel panel. The response shapes were kept identical, so the visitor app needed only new paths rather than a rewrite. Five raw actions were deliberately **not** ported because nothing called them: visitor bookmarks, `set_mode`, `my_guide`, the visitor profile with statistics, and attendance's daily count. Each is a small controller if a screen ever wants one.

B6 · The data model

23 domain tables, plus 8 that belong to the framework (`sessions`, `cache`, `cache_locks`, `jobs`, `job_batches`, `failed_jobs`, `password_reset_tokens`, `migrations`). The full entity-relationship diagrams are in `docs/ERD.md` in the repository.

B6.1 The tables, by the part of the museum they serve

Group	Tables	What it holds
The collection	<code>exhibits</code> , <code>exhibit_translations</code> , <code>exhibit_images</code> , <code>exhibit_training_images</code> , <code>categories</code> , <code>museum_halls</code>	Each exhibit sits in a hall and a category and carries one row per language, a gallery, and a recognition training set
The door	<code>visitors</code> , <code>visit_groups</code>	Who was admitted, as what type, at what fee, cleared by whom. A group is one record and one payment
The visit	<code>scans</code> , <code>attendances</code> , <code>bookmarks</code>	Which exhibits were opened, when a phone entered and left the geofence, what was saved
Guiding and satisfaction	<code>tours</code> , <code>feedback</code> , <code>feedback_answers</code> , <code>survey_questions</code>	Guided tours and the ARTA Client Satisfaction Measurement
Staff	<code>staff</code> , <code>staff_schedules</code> , <code>staff_attendances</code> , <code>staff_attendance_days</code> , <code>attendance_corrections</code>	Accounts and roles, who works when, the daily time record, and corrections with their approval state
Standalone	<code>museum_info</code> , <code>logs</code> , <code>notifications</code>	The museum record (including the fee and the geofence), the audit trail, the notice board. None of these carries a foreign key in either direction

B6.2 Conventions

- **Named primary keys, not a bare id.** Every table uses `exhibit_id`, `visitor_id`, `staff_id` and so on, so a join reads as `scans.exhibit_id = exhibits.exhibit_id` with no ambiguity about which `id` is meant. The same names carry through to the API and the CSV exports.
- **Decimal, not float, for money and coordinates.** `admission_fee`, `total_fee`, `latitude` and `longitude` are decimals, so a peso figure and a geofence edge do not drift.
- **The geofence is data, not a constant.** Its centre and radius are columns on `museum_info`, which is why an Administrator can move the boundary from the panel without a code change.
- **Cascades are deliberate.** Deleting an exhibit removes its translations, images, training images and bookmarks. It does *not* remove its `scans`: visit statistics outlive the exhibit.
- **`bookmarks` has no Eloquent model.** It is a join row with no behaviour, read through the query builder.

B6.3 Three places a foreign key is absent on purpose

Each looks like an oversight until the reason is stated, so each is worth being ready to explain.

What is missing	Why
<code>logs.user_id</code> has no foreign key, and <code>logs.user_name</code> duplicates the name	An audit trail that cascades away when an account is deleted is not an audit trail. The log keeps its own copy of who acted and in what role, so the record of what a since-removed employee did survives their removal. <code>attendances.visitor_name</code> is the same reasoning.
<code>feedback_answers.code</code> duplicates <code>survey_questions.code</code>	A survey answer means whatever the question meant <i>at the time it was answered</i> . Copying the code at submission lets Tourism retire or reword a question without rewriting history, and lets a report still show a retired question's figures. <code>question_id</code> may go null; code does not.
The visitor name columns are repeated on <code>feedback</code>	Feedback can be submitted without signing in. When it is, those columns hold what the respondent typed and <code>visitor_id</code> stays null.

B7 · Three flows worth tracing

A panel will usually ask for one request followed end to end. These three show the most about how the system is put together.

B7.1 A visitor checks in at the door

Visitor phone	POST /api/v1/attendance	GeofenceService	MySQL
-- scan the wall QR			
-- ask the browser for its coordinates			
---- lat, lng, accuracy ---->			
	--- inside the geofence? -->		
		-- read lat/lng	
		and radius	
		from	
		museum_info	
	<-- distance in metres -----		
outside the radius:			
<-- 422 refused, with the distance			
inside:	----- INSERT attendances ----->		
<-- 201 checked in			

Because the centre and radius are rows rather than constants, an Administrator moves the boundary from the panel. This is also why a check-in from home is refused, which is the single most common surprise during testing: the scan appears to “do nothing” when in fact the server has correctly answered that the phone is 40 km away.

B7.2 A visitor identifies an exhibit by camera

```

Visitor phone      Classifier (in the browser)  POST /api/v1/recognition  MySQL
|                 |                         |                         |
|--- camera frame ----->|                 |                 |
|<-- class + confidence ---|                 |                 |
|                 |                         |                 |
|   confident enough:   |                 |                 |
|----- the predicted class name ----->|                 |
|                 |                         |-- resolve to an -->|
|                 |                         exhibit row         |
|<----- exhibit payload -----|                 |
|                 |                         |                 |
|   not confident: keep sampling, or fall back to the QR scanner   |

```

Classification runs on the phone, not the server. The model is roughly 3 MB of static files and the server is only ever asked to resolve a class name to an exhibit row. That is what makes the feature usable on a weak signal, and it is why retraining means regenerating those static files from the training photos staff have uploaded.

B7.3 Tourism exports the ARTA report

```
Tourism      ReportController      ReportBuilder      Exporter (one of four)      MySQL
  |              |              |              |              |
  |-- report, range, format ----->|              |              |
```

```

|          |          |          |--- query, grouped by ARTA ----->| |
|          |          |          |<-- rows ----->|
|          |          |          |-- ReportDataset -->|
|          |          |          |  + sections      |-- read the ---->|
|          |          |          |          letterhead |
|<----- file download ----->|

```

One dataset, four exporters (CSV, XLSX, DOCX, PDF). Adding a format means one new class, not a fifth copy of the query — which is also why the six reports can offer different format sets without any duplicated logic. The PDF exporter alone renders the same Blade layout the browser prints, rather than rebuilding it from flat rows: that layout is the one the museum already checks, signs and files, and a second document that looked almost but not quite like it would be found by whoever had to file it.

B8 · Offline behaviour

The visitor app's service worker keeps the app shell, pictures and audio, and the last good copy of the gated content — the exhibit list, single exhibits, the museum record and the visitor's own clearance.

Rule	Consequence
It is a fallback, not a mode.	The cached copy is served only when the network fails outright. A visitor who never loaded an exhibit while online will not get it offline.
Nothing that changes the museum is cached.	Registration, sign-in, scans, feedback, attendance and the survey all require the server and say so. There is no queue of pending writes to replay: a scan recorded an hour late would put visitors in the wrong hall on the reports.
A live 401 or 403, or signing out, deletes the cache.	This is what stops one visitor's gated content surviving a change of visitor on a shared phone.
Gated content, once on a phone, is on that phone.	A visitor who paid today and then switches to aeroplane mode keeps reading. The fee is a per-visit fee, so this is accepted rather than defended against.
The token is not the worker's business.	It lives in local storage; the service worker does not read it.
Storage belongs to the browser.	iOS evicts caches for sites unused for a week or so, Android less aggressively. A phone returning after a month re-downloads about 5 MB of shell and model.

Cache naming and deployment. Phones cache the app under a name carrying the release hash, which the deploy script writes into the service worker, and the worker file itself is served `no-cache`. A phone opening the app after a deploy therefore installs the new release into a fresh cache and drops the old one, with nothing to do by hand.

Media is fetched on first view, not prefetched at the desk. Pulling every exhibit's picture and a narration per language the moment a visitor is cleared would cost roughly 1.3 MB per exhibit — over 10 MB for the current collection — precisely when a queue is forming at the entrance on the museum's single uplink, and would spend a phone's storage on halls the visitor may never enter. Media is cached on first view instead, which covers the exhibits actually visited without weakening the admission boundary.

B9 · Security controls

What protects the data and where it is enforced. The repository's `docs/SECURITY.md` carries the same list with a file reference on every line.

B9.1 Input and output

- Every string field in every request — panel forms and API JSON bodies alike — has HTML tags and control characters removed **before validation** (`SanitizeInput`). Passwords are exempt, deliberately.
- All database access is parameterised: Eloquent throughout, panel and API alike. Request shapes are Form Request classes.
- Output is escaped: Blade's `{{ }}` everywhere in the panel — the single raw echo renders a hard-coded SVG icon set — and an explicit escaping helper in the visitor app.
- Uploaded files are served through routes that take the `basename()` of the filename and confirm the file exists, never by a user-supplied path.
- CSRF tokens on every form; an expired one is caught and explained in plain language rather than shown as a framework error.

How to demonstrate it: register a visitor named `<script>alert(1)</script>`. The record saves as `alert(1)` and nothing executes anywhere.

B9.2 Browser and API hardening

Control	Detail
Response headers (panel)	Content-Security-Policy (self plus the two CDNs the layout uses), X-Frame-Options: DENY, nosniff, Referrer-Policy, Permissions-Policy scoping camera and geolocation to this origin only, X-Powered-By removed
Response headers (API)	nosniff, Cross-Origin-Resource-Policy: same-origin, Cache-Control: no-store — except the exhibit list, which is private, no-cache with an ETag so an unchanged museum is a 304
CORS	Only origins listed in <code>API_ALLOWED_ORIGINS</code> . Same-origin needs none, and the list is empty in production
Proxy trust	Explicit: loopback and Cloudflare's published ranges, so X-Forwarded-For cannot be spoofed to dodge the per-address limits
Debug output	<code>APP_DEBUG</code> is forced off in production even if the configuration says otherwise , so no stack trace, file path or query ever reaches a browser. Custom pages for 401, 403, 404, 419, 429, 500 and 503

Rate limits

Scope	Limit
Visitor API, overall	60 requests per minute per address
Camera recognition sampling	120 per minute
Visitor registration	10 per minute
Visitor sign-in	6 per minute per account; 30 per 5 minutes per address
Staff sign-in	5 failed attempts per address per minute
The panel, per account	300 per minute

Scope	Limit
The two routes that spend money (AI translate and narrate)	20 per minute and 300 per day, per staff account

B9.3 Logging and alerting

Record	Retention	Contents
Audit log (logs table)	Kept in the database	Every POST/PUT/PATCH/DELETE by a signed-in account: who, what, from where, when. Exportable as CSV or PDF by Tourism only
Security log	90 days	Every failed sign-in, logout, role denial, sign-in and sign-out
Application log	30 days, rotated daily	Framework and application events
Alerts by email	Throttled to one per problem per 30 minutes	Any unhandled error in production, a failed backup, or a backup older than 36 hours

The backup watchdog runs off ordinary panel traffic, not off the scheduler. That is the point: it still fires when the scheduler is the thing that broke. A watchdog that depended on the scheduler would go quiet at exactly the moment it was needed.

B9.4 Data at rest and in transit

- **In transit:** TLS at the edge or on the web server; secure cookies.
- **Credentials and tokens:** never stored in the clear — bcrypt for passwords, SHA-256 for visitor tokens.
- **Session payloads:** encrypted.
- **Backups: encrypted** with XChaCha20-Poly1305 (libsodium secretstream). The construction is authenticated, so a tampered file refuses to decrypt rather than yielding corrupted rows. A backup that goes missing on a USB stick is then a lost stick, not a breach.
- **Visitor personal data is stored in the clear in MySQL** — see B15, where it is recorded as a known limitation rather than hidden.

B9.5 Dependencies and delivery

`composer audit` and `npm audit` run on every push alongside the full test suite. Dependabot opens dependency-update pull requests weekly, which CI tests the same way. Deployment is blue/green with one-command rollback, and every deploy takes a database backup before migrating and health-checks the new release before it goes live (B12).

B9.6 Security review record

A review of the full git history for committed keys and credentials found none. Findings that were found and fixed, recorded here because a review with no findings is not a review:

- Directory listings were on — the disabling directive sat inside a conditional block for a web-server module the development machine does not load.
- The demonstration data seeder was minting Administrator accounts with a known password.
- Two HTML sinks for staff-typed captions and fun facts were unescaped.

- The offline API cache survived a change of visitor on a shared phone.
- `X-Frame-Options` was being sent with two different values.

The boundaries these touched are now pinned by tests (`tests/Feature/Api/AuthBoundaryTest.php`), so a regression fails the build rather than waiting for the next review.

B10 · Installing it locally

Requires PHP 8.2+ with `mbstring` `gd` `sodium` `pdo_mysql` `zip`, Composer, Node 20+, and MySQL 8.

```
composer install
npm ci
cp .env.example .env          # DB_* for a local MySQL; the rest can wait
php artisan key:generate
php artisan migrate --seed    # schema, the ARTA survey, sample exhibits with pictures
npm run build                 # the panel's stylesheet
php artisan serve              # http://127.0.0.1:8000
```

The seeder prints two starting logins — one Administrator and one Tourism Head — once. Both must change their password at first sign-in. Set `SEED_ADMIN_PASSWORD` and `SEED_TOURISM_PASSWORD` beforehand to choose them.

The visitor app is at `/visitor/`. **On a phone it needs HTTPS** for the camera, the service worker and location: use a tunnel to the local server, or test those parts against the deployed site. `VISITOR_GEOFENCE=false` and `ATTENDANCE_GEOFENCE=false` let a developer exercise the entry flow and staff clock-in from a desk that is not in Baler; **production ignores both and always runs the fences.**

On Laragon: `extension=zip` ships commented out in `php.ini`. Uncomment it and restart Apache, or *QR Codes* → *Download All* fails. Single downloads and *Print All* still work without it.

B11 · Configuration reference

Everything lives in `.env`, which is never inside a release directory. `.env.example` documents every key with a comment. The ones that matter operationally:

Key	What it decides
APP_ENV, APP_DEBUG	<code>production</code> and <code>false</code> on a server. Both geofences and the development CORS origins switch on with the first; debug output is forced off in production regardless, but set it anyway
APP_URL, APP_TIMEZONE	The public HTTPS address; <code>Asia/Manila</code> , which both database connections pin MySQL's session zone to, so a server whose MySQL runs in UTC still dates a survey filed at 1 a.m. on the right day
DB_*	A MySQL user scoped to this one database
SESSION_SECURE_COOKIE	<code>true</code> once TLS is in place
BCRYPT_ROUNDS	12
API_ALLOWED_ORIGINS	Leave empty unless a front end on another origin exists. Same-origin needs nothing
GEMINI_API_KEY	Optional. Without it the AI buttons say so and nothing else changes
GEMINI_TEXT_MODEL, GEMINI_TTS_MODEL	Override when Google retires a model — the defaults name preview models, which will go stale
BACKUP_ENCRYPTION_KEY	The most important key in the file. Generated once by <code>php artisan db:backup:key</code> ; a copy must be kept off the machine
BACKUP_COPY_TO, BACKUP_KEEP	A second disk or synced folder; how many nights to keep (14)
MYSQLDUMP_PATH	Only if <code>mysqldump</code> is not on the PATH
MAIL_*, ALERT_EMAIL	Real values, or nobody hears when a backup fails
ALERT_BACKUP_MAX_AGE_HOURS	36, the age at which a dump counts as stale
ATTENDANCE_GEOFENCE, VISITOR_GEOFENCE	Development-only escape hatches. Ignored in production. Leave unset on a server

B12 · Deployment, rollback and CI

B12.1 The layout on the server

```

/var/www/museobaler/
  current -> releases/20260919143000 <- the web server's docroot is current/public
  releases/
    20260918090000/ <- the previous release, kept for rollback
    20260919143000/ <- live
  shared/
    .env <- secrets; never inside a release
    storage/ <- logs, sessions, backups
    public/ <- uploaded media and the trained model
      images/{exhibits,training,branding}
      audio/
      visitor/model/
  PREVIOUS <- path of the release before this one

```

Every release is a complete, self-contained copy of the application. **The only thing that changes at go-live is where the `current` symlink points** — that is the whole of the blue/green idea. The new copy is built, migrated and health-checked while the old one keeps serving, and the switch is one atomic rename.

B12.2 What a deploy does, and where it stops

Step	Action	If it fails
1	Clone the exact ref into a new <code>releases/<timestamp></code>	Nothing changed
2	Link the shared <code>.env</code> , storage and media into it	Nothing changed
3	<code>composer install --no-dev, npm ci && npm run build</code> , cache config/routes/views	Nothing changed
4	Take a database backup , then migrate <code>--force</code>	Old code still live; restore the dump if a migration half-applied
5	Boot the new release on a private port and require a 200 from <code>/up</code>	Old code still live
6	Atomically move <code>current</code> ; reload PHP-FPM	—
7	Delete releases older than the last three	—

Nothing a visitor or staff member can see changes until step 6, and step 6 is a single rename. There is no maintenance window.

B12.3 Rolling back

`deploy/rollback.sh` moves `current` back to the release recorded in `PREVIOUS` and reloads PHP-FPM. It takes about a second, and running it again goes forward again.

The database is deliberately not touched. Migrations in this project only add columns and tables, so the previous code runs correctly against the newer schema. If a migration itself did damage, the dump taken at step 4 is on disk — but restoring it is a decision a person makes, not something a rollback script should do on its own.

B12.4 What CI guarantees before any of that

On every push: the full test suite on in-memory SQLite (the visitor API included), `composer audit` and `npm audit`. Deploy only from a ref that is green.

B12.5 The scheduler must be woken by the operating system

```
# Linux
* * * * * cd /var/www/museobaler/current && php artisan schedule:run >> /dev/null 2>&1

# Windows, once, from an elevated PowerShell
powershell -ExecutionPolicy Bypass -File deploy\windows\install-scheduler-task.ps1
```

Nothing runs at 02:30 unless this is done. A freshly installed machine has no backups at all until the scheduler is registered. This is the most commonly missed step in the whole setup.

B13 · Backup and restore

B13.1 What runs nightly

At 02:30, `php artisan db:backup` takes a `mysqldump --single-transaction`, gzips it, encrypts it, keeps 14, and copies it to `BACKUP_COPY_TO` on a second disk, with an email alert on any failure. **The same run writes a media archive beside the dump** — every uploaded picture, training photo and audio guide — encrypted with the same key and kept for the same fortnight. A database backup without the media is only half a restore.

B13.2 The commands

```
php artisan db:backup:key                # once, at setup. Put the output in .env
                                         # AND keep a copy off the machine

php artisan db:backup                    # a dump + a media archive

php artisan db:backup:decrypt storage/app/backups/museobaler-<stamp>.sql.gz.enc
gunzip -c museobaler-<stamp>.sql.gz | mysql museobaler

php artisan db:backup:decrypt ...-media.tar.gz.enc
tar -xzf ...-media.tar.gz -C public
```

B13.3 There is deliberately no `db:restore` command

Restoring is two documented commands run by a person who has decided to run them. An automated restore that could wipe the live database by accident was judged worse than a checklist, and `docs/RESTORE.md` is written for whoever is holding the drive on a bad morning — in the order the steps have to happen, database first, then media.

The encryption key. If `BACKUP_ENCRYPTION_KEY` is lost, every backup ever made with it is permanently unreadable — by anyone, including the authors of this system. It must exist in two places that cannot fail together: in `.env` on the server, and somewhere that is not the server. A copy on the same machine, the same drive or the same cloud account as the backups does not count. If the key is ever rotated, **keep the old one too, labelled with dates** — old backups still need it.

B13.4 Restore drills

A backup nobody has restored is a hope, not a backup. The quarterly drill is a scheduled item in Part A, section A15.2, and the restore document has a table to record the date each drill was done.

B14 · Verification

All figures below were produced by running the suites at the time this manual was written, not quoted from earlier notes.

```
php artisan test
  Tests:    406 passed (4902 assertions)
  Duration: 84.14s

npm test
  tests 15    pass 15    fail 0
```

B14.1 What the suites cover

Suite	Covers
tests/Feature/	The panel end to end: role access in both directions, the admission gate, the front desk and group pricing, attendance and its corrections, reports and every export format, security headers, production configuration, and the desktop-only boundary
tests/Feature/Api/	The visitor API: registration and sign-in, the clearance gate, exhibits, scans, feedback, and the authorisation boundaries between one visitor and another
tests/Unit/	Pure logic: the geofence, the password policies, the ARTA survey definitions
tests/js/	The service worker's routing rules — what is cached, what must never be

The suite runs on SQLite; production is MySQL. Anything MySQL-specific — a raw `NOW()`, an enum change — needs a manual check against MySQL. `php artisan migrate:fresh --seed` against a scratch database name is the quickest way. This is a stated trade-off: in-memory SQLite is what makes a 406-test suite finish in 84 seconds on every push.

B15 · Known limitations and design boundaries

What the system deliberately does not do, and what will trip a developer who assumes otherwise. Each of these is a decision, not an omission.

B15.1 Deliberate boundaries

Boundary	Reason, and what changing it would take
The floor plan is a drawing in the code	The walls, rooms and stairs are two inline SVG drawings, duplicated in the panel's map view and the visitor app. Exhibit <i>pins</i> are data, dragged and saved by staff. The building does not change; a drawing edited once in a vector tool looks better than a floor-plan editor nobody would use twice. If a wall moves, edit both drawings keeping the same <code>viewBox</code> so existing pins stay put.
AI generation is synchronous	Each request blocks the browser for a few seconds. Queueing was considered and deferred: it would need a queue worker running on the server — one more thing to keep alive — for a task that happens a few times a month.
Visitor personal data is stored in the clear	Names, emails, ages and addresses are ordinary columns because the panel, the API and every report read them; column-level encryption would make search, grouping and the CSV exports decrypt row by row. Mitigated by a scoped database user, no direct database exposure, encrypted backups , and disk encryption on the host. Recorded here rather than hidden.
One museum, one currency, one time zone	The time zone is pinned, the fee is in pesos, and the barangay list is Baler's thirteen. Running this for a second museum means a second installation, not a setting.
No visitor password reset	No mail is sent to visitors and there is no identity to verify against. Email is checked for shape and a live domain, not ownership — a verification email in the entrance queue was judged a worse trade than the occasional typo.
No real-time dashboards	The notification bell polls every ten seconds and the desk's list refreshes on action. WebSockets would need a socket server.
No native app	The PWA installs from the browser and uses the camera and GPS. A store listing would add a review process for no added capability.

B15.2 Things a developer will trip over

- **ext-zip must be enabled** for *Download All (ZIP)*. Laragon ships it commented out.
- **There is no `db:restore` command**, by design (B13.3).
- **The scheduler must be woken by the OS**, or nothing is ever backed up (B12.5).
- **Tests run on SQLite, production on MySQL** (B14.1).
- **Tests write QR files into `public/images/qr/`**. That folder is ignored by git, so nothing leaks into a commit, but the files do appear.
- **Pint is not a CI gate**. The codebase aligns its array arrows throughout and Pint would rewrite about 150 files. CI runs tests and audits only.
- **The notifications table has no editor**. The app reads it; nothing in the panel writes it. It is seed data until someone builds the notice-board screen.
- **Gemini's default model names will go stale**. The defaults name preview models; set the override keys when Google retires one.

B16 · Data privacy (RA 10173)

Question	Answer
What is collected	From visitors: name, age, sex, email address, city/barangay/province/country, visit type, and — only while on the grounds — the phone's location for the attendance geofence. Nothing else.
Why	The logbook the museum is required to keep; admission, because the local/tourist distinction decides the fee; and the ARTA Client Satisfaction Measurement the LGU files.
Who can see it	The two staff roles in Part A, section A2.1, and nobody else. No third-party service receives visitor data. The AI translation feature sends exhibit text only — never a visitor record.
Location tracking	The system records that a phone entered and left a circle around the museum. It does not track position inside the building, and it records nothing outside that circle.
Retention	Records are kept for reporting. A <code>museum:clear-records</code> command purges by date if a retention policy is adopted; no automatic deletion happens without one.
Security measures	Role-based access enforced server-side, no direct database exposure, a scoped database user, encrypted sessions, hashed credentials, encrypted off-machine backups, and a full audit trail of staff actions. See B9.

A P P E N D I C E S

Reference

Command reference, the directory map, the checklist that must be completed before the system carries real visitors, and a glossary of the terms used throughout this manual.

Appendix A · Command reference

A.1 Project commands

Command	What it does
<code>php artisan db:backup</code>	An encrypted database dump plus a media archive into the backup folder, pruned to the configured count. <code>--keep=</code> overrides that count. This is what runs nightly
<code>php artisan db:backup:key</code>	Generates the backup encryption key. Run once, at setup. Put the output in <code>.env</code> and a copy somewhere that is not the server
<code>php artisan db:backup:decrypt <file></code>	Decrypts one dump or one media archive, for a restore
<code>php artisan reports:export</code>	Produces a report file without a browser — for a scheduled or unattended export
<code>php artisan exhibits:thumbs</code>	Rebuilds exhibit image derivatives. <code>--force</code> rebuilds ones already current
<code>php artisan museum:clear-records</code>	Purges visitor records by date, if a retention policy is adopted. Nothing calls it automatically
<code>php artisan staff:password</code>	Sets a staff password from the command line — the recovery path when nobody can sign in to issue one

A.2 Everyday framework commands

Command	What it does
<code>php artisan migrate --seed</code>	Builds the schema and seeds the ARTA survey and sample exhibits
<code>php artisan migrate:fresh --seed</code>	Rebuilds from nothing. Never against production
<code>php artisan test</code>	The PHPUnit suite (406 tests)
<code>npm test</code>	The service-worker suite (15 tests)
<code>npm run build</code>	The panel's stylesheet
<code>php artisan schedule:run</code>	What cron or Task Scheduler calls every minute

Appendix B · Directory map

app/	
Console/Commands/	backup, backup key, decrypt, report export, clear records, thumbnails, set staff password
Http/Controllers/	the panel (18)
Http/Controllers/Api/	the visitor API (10)
Http/Middleware/	the 11 gates described in B3
Models/	Eloquent (23); Visitor carries the door's logic
Rules/	custom validation rules
Services/	Gemini, geofence, recognition, image search, QR
Support/	the ARTA survey, password policies, backups, alerts
Support/Reports/	ReportBuilder plus the four exporters
bootstrap/, config/	framework wiring; the trusted proxy list
database/	
migrations/	51 migrations
seeders/media/	the seeded exhibits' pictures and audio
deploy/	deploy.sh, rollback.sh, the Windows scheduler task
docs/	architecture, ERD, security, deployment, hosting, restore, ownership, limitations, staff manual
public/	
visitor/	the visitor app: index.html, js/app.js, sw.js, model/
images/, audio/	uploads (ignored by git; shared across releases)
resources/views/	Blade: the panel, the print layouts, the mail
routes/	
web.php	the panel
api.php	the visitor API
console.php	the scheduled tasks
tests/	
Feature/	the panel
Feature/Api/	the visitor API
Unit/	pure logic
js/	the service worker

Appendix C · Pre-production checklist

What the code cannot do for itself. Every item is a line in `.env` or in the web server configuration. Automated tests cover only the first of these, so the rest is a human checklist by necessity.

✓	Item	Why it matters
☐	HTTPS everywhere	Without TLS the visitor app has no camera, no location and no offline cache, and the staff session cookie travels in the clear. Then set <code>SESSION_SECURE_COOKIE=true</code> and enable HSTS once every subdomain is on TLS
☐	APP_ENV=production, APP_DEBUG=false	Both geofences and the development CORS origins switch correctly on the first; debug output is forced off in production anyway, but set it
☐	CORS left empty	Leave <code>API_ALLOWED_ORIGINS</code> empty unless a front end on another origin exists
☐	Trusted proxy ranges current	If the origin is reached through a CDN edge, the trusted ranges must be current or every visitor shares one address for rate limiting
☐	ext-zip enabled	Required for the bulk QR download
☐	mysqldump reachable	On the PATH, or <code>MYSQLDUMP_PATH</code> set — or there are no backups
☐	The scheduler woken every minute	Cron on Linux, Task Scheduler on Windows. Nothing runs nightly without it
☐	BACKUP_ENCRYPTION_KEY set, and copied off the machine	See B13.3. This is the item with no second chance
☐	BACKUP_COPY_TO on a different disk	A backup on the disk that fails is not a backup
☐	ALERT_EMAIL and real MAIL_*	So someone hears when a backup fails. And someone must actually read that inbox
☐	Directory listings off	Off in the shipped Apache configuration; on nginx, <code>autoindex</code> defaults to off — do not turn it on
☐	Protected media denied at the web server	On nginx, deny <code>/images/exhibits/</code> and <code>/audio/</code> before the generic static-file rule, or the web server serves gated files without consulting the gate
☐	The demonstration seeder never run on production	It refuses to run there, but do not make it try
☐	The geofence pin set from a phone at the entrance	Not from a desktop. See the warning in Part A, section A9.2
☐	Both seeded passwords changed	Both accounts are forced to change at first sign-in; confirm both have changed
☐	One restore drill completed	Before the system carries records that matter, not after

Appendix D · Glossary

Term	Meaning in this system
Administrator	The museum staff role: the front desk, the collection, the map, tours, museum settings, and clocking in. Despite the name it is <i>not</i> the most powerful role — it cannot create accounts or read the audit export
Admission gate	The server-side rule that releases exhibit content only after the desk has taken the fee or sighted a residency ID. The app's waiting screen renders its answer
ARTA / CSM	The Anti-Red Tape Authority's Client Satisfaction Measurement — the visitor survey the LGU is required to file
Blue/green deploy	Building and testing a complete new copy of the application beside the running one, then switching between them with a single atomic rename
Cleared	A visitor whose fee has been collected or whose Baler residency ID has been sighted. Only a cleared visitor's phone can load exhibit content
DTR	Daily Time Record — the monthly attendance sheet for one staff member
Geofence	A circle around the museum, defined by a centre and a radius stored in the database, used both to count a visitor as present and to permit a staff clock-in
Group code	A six-letter code issued when a party is registered, valid the same day only and only for the headcount entered. It contains no letter O or I
Kiosk code	The QR code on the staff-room tablet that staff scan to clock in. It rotates every 60 seconds, so a photograph of it is useless
PWA	Progressive Web App — a website that installs itself onto a phone's home screen from the browser, with no app store involved
Service worker	The part of the visitor app that keeps a local copy of what the phone has already loaded, so a dropped signal mid-tour does not empty the screen
Storyline mode	The guided path through the collection, in curated order, with the next stop highlighted on the map. The alternative is Explore Freely
Tourism Head	The Municipal Tourism Office role: staff accounts, schedules, correction approvals, the survey question bank, and the audit export
Signed URL	A time-limited link carrying its own proof of authorisation, used for exhibit images and audio because an image element cannot send an authorisation header

Appendix E · Further documentation in the repository

File	Written for
docs/STAFF_MANUAL.md	Museum staff and the Tourism office — the day-to-day job, no technical background assumed
docs/ARCHITECTURE.md	How the pieces fit: context, the three faces, the request pipeline, deployment, what is generated
docs/ERD.md	The database in full, with the entity-relationship diagrams and the joins that are absent on purpose
docs/SECURITY.md	Every control, with the file that enforces it, plus the Data Privacy Act notes
docs/DEPLOYMENT.md	First-time server setup, deploying, rolling back
docs/HOSTING.md	Buying the hosting, setting it up from scratch, handing the bill to the museum
docs/RESTORE.md	Whoever is holding the backup drive on a bad morning
docs/OWNERSHIP.md	Who holds which account, credential and key — reviewed quarterly
docs/LIMITATIONS.md	What the system deliberately does not do, and what will trip a developer

— End of manual —